

ບົດທີ 1: ຄວາມຮູ້ເບື້ອງຕົ້ນກ່ຽວກັບວິສະວະກຳຊອບແວ (Introduction to Software Engineering)

1.1 ຊອບແວແມ່ນຫຍັງ? ແລະ ວິທີການຊອບແວໃນອະດີດ

ກ. ຊອບແວແມ່ນຫຍັງ? (What is Software?)

ໃນຍຸກດິຈິຕອນ, ຄຳວ່າ "ຊອບແວ" ເປັນຄຳທີ່ເຮົາຮູ້ເຄີຍກັນດີ. ແຕ່ໃນທາງວິສະວະກຳ, ຊອບແວບໍ່ແມ່ນພຽງແຕ່ໂປຣແກຣມ ຫຼື ແອັບພລິເຄຊັນທີ່ເຮົາໃຊ້ໃນຄອມພິວເຕີ ແລະ ໂທລະສັບມືຖືເທົ່ານັ້ນ. ຊອບແວປະກອບດ້ວຍ 3 ອົງປະກອບຫຼັກຄື:

- ຊຸດຄໍາສັ່ງ (Instructions):** ຄືສ່ວນຂອງໂປຣແກຣມທີ່ຂຽນຂຶ້ນດ້ວຍພາສາຄອມພິວເຕີຕ່າງໆ (ເຊັ່ນ Java, Python, C++) ເພື່ອສັ່ງໃຫ້ຮາດແວ (Hardware) ເຮັດວຽກຕາມທີ່ຕ້ອງການ, ຕັ້ງແຕ່ການປະມວນຜົນຂໍ້ມູນໄປຈົນເຖິງການສະແດງຜົນເທິງໜ້າຈໍ.
- ໂຄງສ້າງຂໍ້ມູນ (Data Structures):** ຄືວິທີການຈັດລະບຽບ ແລະ ເກັບຮັກສາຂໍ້ມູນ ເພື່ອໃຫ້ຊຸດຄໍາສັ່ງສາມາດເຂົ້າເຖິງ ແລະ ຈັດການຂໍ້ມູນໄດ້ຢ່າງມີປະສິດທິພາບ.
- ເອກະສານປະກອບ (Documentation):** ຄືເອກະສານທຸກຢ່າງທີ່ກ່ຽວຂ້ອງກັບຊອບແວ, ເຊິ່ງລວມມີທັງເອກະສານສໍາລັບຜູ້ໃຊ້ (ເຊັ່ນ: ຄູ່ມືການໃຊ້ງານ) ແລະ ເອກະສານສໍາລັບນັກພັດທະນາ (ເຊັ່ນ: ເອກະສານອອກແບບລະບົບ, ຄອມເມັນໃນໂຄດ).

ລັກສະນະພິເສດຂອງຊອບແວ:

ສິ່ງທີ່ເຮັດໃຫ້ຊອບແວແຕກຕ່າງຈາກຜະລິດຕະພັນອື່ນໆ (ເຊັ່ນ: ລົດໃຫຍ່, ຕັ້ງ) ຄື:

- ຊອບແວຖືກ "ພັດທະນາ" ຫຼື "ສ້າງຂຶ້ນທາງວິສະວະກຳ", ບໍ່ໄດ້ຖືກ "ຜະລິດ" ໃນແບບອຸດສາຫະກຳ: ເຮົາບໍ່ສາມາດສ້າງສາຍພານການຜະລິດຊອບແວແບບໂຮງງານໄດ້. ຊອບແວແຕ່ລະໂຕມີຄວາມຊັບຊ້ອນ ແລະ ລັກສະນະສະເພາະຕົວ.
- ຊອບແວບໍ່ມີການ "ລຸ່ຍຫຼຽນ" ທາງກາຍະພາບ: ຊອບແວບໍ່ເສື່ອມສະພາບຄືກັບເຄື່ອງຈັກ, ແຕ່ມັນສາມາດ "ຫຼ້າສະໄໝ" ໄດ້. ເມື່ອເທັກໂນໂລຢີປ່ຽນ, ຄວາມຕ້ອງການຂອງຜູ້ໃຊ້ປ່ຽນ, ຊອບແວທີ່ບໍ່ໄດ້ຮັບການປັບປຸງກໍຈະກາຍເປັນສິ່ງທີ່ບໍ່ມີປະໂຫຍດ.

ຂ. ວິທີການຊອບແວໃນອະດີດ (The Software Crisis)

ໃນຊ່ວງປີ 1960s ຫາ 1980s, ໂລກຄອມພິວເຕີໄດ້ພັດທະນາໄປຢ່າງກ້າວກະໂດດ. ຮາດແວມີລາຄາຖືກລົງ ແລະ ມີປະສິດທິພາບສູງຂຶ້ນຢ່າງມະຫາສານ. ສິ່ງນີ້ເຮັດໃຫ້ເກີດຄວາມຕ້ອງການຊອບແວທີ່ມີຂະໜາດໃຫຍ່ແລະຊັບຊ້ອນຫຼາຍຂຶ້ນ ເພື່ອຄວບຄຸມທຸກຢ່າງ, ຕັ້ງແຕ່ລະບົບທະນາຄານ, ການບິນ, ໄປຈົນເຖິງໂຄງການອະວະກາດ.

ແຕ່ໜ້າເສຍດາຍ, ວິທີການ "ສ້າງ" ຊອບແວໃນຕອນນັ້ນຍັງຄົງເປັນແບບ "ສິລະປະ" ທີ່ຂຶ້ນກັບຄວາມສາມາດສ່ວນບຸກຄົນຂອງໂປຣແກຣມເມີ, ບໍ່ມີຂະບວນການ, ບໍ່ມີມາດຕະຖານ, ແລະ ບໍ່ມີວິທີການຈັດການທີ່ເປັນລະບົບ. ສະຖານະການນີ້ໄດ້ນໍາໄປສູ່ສິ່ງທີ່ເອີ້ນວ່າ "ວິທີການຊອບແວ" (The Software Crisis), ເຊິ່ງມີລັກສະນະດັ່ງນີ້:

- ໂຄງການເກີນງົບປະມານ (Over Budget): ຕົ້ນທຶນໃນການພັດທະນາສູງກວ່າທີ່ປະເມີນໄວ້ຫຼາຍເທົ່າ.
- ໂຄງການຊັກຊ້າກວ່າກໍານົດ (Over Schedule): ບໍ່ສາມາດສົ່ງມອບຊອບແວໄດ້ຕາມເວລາທີ່ກໍານົດ.
- ຊອບແວບໍ່ມີຄຸນນະພາບ (Low Quality): ເຕັມໄປດ້ວຍ Bug, ເຮັດວຽກຜິດພາດຕະຫຼອດ, ແລະ ບໍ່ໜ້າເຊື່ອຖື.

- **ຊອບແວບໍ່ຕອບໂຈດຄວາມຕ້ອງການຂອງຜູ້ໃຊ້ (Doesn't Meet User Needs):** ເມື່ອສ້າງສໍາເລັດ, ຜູ້ໃຊ້ກໍພົບວ່າມັນບໍ່ສາມາດເຮັດໃນສິ່ງທີ່ພວກເຂົາຕ້ອງການໄດ້ແທ້ໆ.
- **ການບໍາລຸງຮັກສາເປັນໄປໄດ້ຍາກ (Hard to Maintain):** ໂຄດທີ່ຂຽນຂຶ້ນບໍ່ມີລະບຽບ, ເຮັດໃຫ້ການແກ້ໄຂ Bug ຫຼື ເພີ່ມ Feature ໃໝ່ ເປັນເລື່ອງທີ່ຍາກລຳບາກ ແລະ ມີຄ່າໃຊ້ຈ່າຍສູງ.

ຕົວຢ່າງຈາກປະຫວັດສາດ:

- **ລະບົບປະຕິບັດການ IBM System/360 (OS/360):** ຖືກພິຈາລະນາເປັນໂຄງການຊອບແວທີ່ໃຫຍ່ທີ່ສຸດໃນຍຸກນັ້ນ, ໃຊ້ໂປຣແກຣມເມີຫຼາຍພັນຄົນ ແລະ ໃຊ້ເວລາພັດທະນາຫຼາຍປີ. ແຕ່ຜົນທີ່ໄດ້ຮັບຄື ໂຄງການຊັກຊ້າກວ່າກໍານົດ, ງົບປະມານບານປາຍ, ແລະ ເມື່ອປ່ອຍອອກມາແລ້ວກໍຍັງເຕັມໄປດ້ວຍ Bug ຈຳນວນມະຫາສານ. ໂຄງການນີ້ເປັນກໍລະນີສຶກສາຄລາສສິກຂອງວິກິດການຊອບແວ.
- **ເຄື່ອງສາຍລັງສີປິ່ນປົວມະເຮັງ Therac-25 (1985-1987):** ນີ້ຄືຕົວຢ່າງທີ່ໂສກເສົ້າທີ່ສຸດ. Bug ໃນຊອບແວຄວບຄຸມເຄື່ອງສາຍລັງສີ ເຮັດໃຫ້ເຄື່ອງປ່ອຍລັງສີໃນປະລິມານທີ່ສູງເກີນຂະໜາດ, ສົ່ງຜົນໃຫ້ມີຄົນເຈັບເສຍຊີວິດ ແລະ ບາດເຈັບສາຫັດຫຼາຍລາຍ. ເຫດການນີ້ສະແດງໃຫ້ເຫັນວ່າ Bug ໃນຊອບແວສາມາດເປັນອັນຕະລາຍເຖິງຊີວິດໄດ້.
- **ລະບົບຄວບຄຸມການຈະລາຈອນທາງອາກາດ:** ໃນຍຸກນັ້ນ, ຫຼາຍໆໂຄງການທີ່ພະຍາຍາມສ້າງລະບົບຄວບຄຸມການບິນໃໝ່ໆ ຕ້ອງລົ້ມເຫຼວ ຫຼື ປະເຊີນກັບບັນຫາຊັກຊ້າ ແລະ ງົບປະມານບານປາຍ ເນື່ອງຈາກຄວາມຊັບຊ້ອນຂອງລະບົບ.

ບົດສະຫຼຸບ: ຈຸດກໍານົດຂອງວິສະວະກໍາຊອບແວ

ວິກິດການຊອບແວໄດ້ເຮັດໃຫ້ຄົນໃນວົງການຄອມພິວເຕີທົ່ວໂລກຄຳນຶງວ່າ ການສ້າງຊອບແວທີ່ຊັບຊ້ອນນັ້ນ ບໍ່ສາມາດອາໄສພຽງແຕ່ "ການຂຽນໂຄດ" ແບບຕາມໃຈໄດ້ອີກຕໍ່ໄປ. ມັນຕ້ອງການ **"ວິທີການທີ່ເປັນລະບົບ, ມີລະບຽບວິໃນ ແລະ ສາມາດວັດຜົນໄດ້"** ຄືກັນກັບການສ້າງຂົວ, ສ້າງຕຶກ ຫຼື ສ້າງເຄື່ອງຈັກ.

ຈາກຄວາມຕ້ອງການນີ້, ສາດວິຊາທີ່ເອີ້ນວ່າ **"ວິສະວະກໍາຊອບແວ (Software Engineering)"** ຈຶ່ງໄດ້ກໍານົດຂຶ້ນ. ມັນຄືການນຳເອົາຫຼັກການທາງວິສະວະກໍາມາປະຍຸກໃຊ້ກັບທຸກຂັ້ນຕອນຂອງການພັດທະນາຊອບແວ, ເພື່ອຮັບປະກັນວ່າເຮົາຈະສາມາດສ້າງຜະລິດຕະພັນທີ່ມີຄຸນນະພາບ, ສົ່ງມອບໄດ້ທັນເວລາ ແລະ ຢູ່ໃນງົບປະມານທີ່ກໍານົດ. ສິ່ງທີ່ພວກເຮົາຈະໄດ້ຮຽນໃນວິຊານີ້ ກໍຄືຫຼັກການ ແລະ ຂະບວນການເຫຼົ່ານັ້ນເອງ.

1.2 ວິສະວະກໍາຊອບແວແມ່ນຫຍັງ? ຄວາມແຕກຕ່າງຈາກວິທະຍາສາດຄອມພິວເຕີ ແລະ ການຂຽນໂປຣແກຣມ

ຈາກວິກິດການຊອບແວທີ່ໄດ້ກ່າວມາ, ເຮົາເຫັນໄດ້ວ່າການພຽງແຕ່ "ຂຽນໂປຣແກຣມ" ໄດ້ນັ້ນ ບໍ່ພຽງພໍທີ່ຈະສ້າງລະບົບຊອບແວຂະໜາດໃຫຍ່ ແລະ ມີຄຸນນະພາບ. ສິ່ງນີ້ໄດ້ນຳໄປສູ່ການກໍານົດຂອງສາດວິຊາ **"ວິສະວະກໍາຊອບແວ"**. ເພື່ອໃຫ້ເຂົ້າໃຈຢ່າງແທ້ຈິງ, ເຮົາຈຳເປັນຕ້ອງສາມາດໄຈ້ແຍກລະຫວ່າງ 3 ຄຳສັບທີ່ມັກຈະຖືກໃຊ້ປົນກັນຄື: ການຂຽນໂປຣແກຣມ, ວິທະຍາສາດຄອມພິວເຕີ ແລະ ວິສະວະກໍາຊອບແວ.

ກ. ການປຽບທຽບໂດຍຫຍໍ້

ເພື່ອໃຫ້ເຫັນພາບ, ໃຫ້ເຮົາລອງປຽບທຽບໃສ່ໂລກຂອງການກໍ່ສ້າງ:

ສາດວິຊາ	ສິ່ງທີ່ເນັ້ນ (Focus)	ປຽບທຽບໃສ່ການກໍ່ສ້າງ
ການຂຽນໂປຣແກຣມ (Programming)	"ການລົງມືເຮັດ" - ການຂຽນໂຄດເພື່ອສ້າງສິ່ງໃດໜຶ່ງ.	ຄືກັບ "ຊ່າງໄມ້" ຫຼື "ຊ່າງກໍ່ອິດ" ຜູ້ທີ່ມີທັກສະ ໃນການໃຊ້ເຄື່ອງມືເພື່ອສ້າງສ່ວນປະກອບ ຕ່າງໆ ຂອງບ້ານ.
ວິທະຍາສາດຄອມພິວເຕີ (Computer Science)	"ທິດສະດີ ແລະ ຫຼັກການ" - ການສຶກສາພື້ນຖານວ່າຄອມພິວເຕີ ເຮັດວຽກແນວໃດ.	ຄືກັບ "ນັກຟີຊິກ" ຫຼື "ນັກວັດສະດຸສາດ" ຜູ້ທີ່ ສຶກສາຄຸນສົມບັດຂອງໄມ້, ຄວາມແຂງແຮງ ຂອງຊີເມັນ, ຫຼື ກົດເກນທາງຟີຊິກ.
ວິສະວະກຳຊອບແວ (Software Engineering)	"ຂະບວນການ ແລະ ການຈັດການ" - ການນຳໃຊ້ທິດສະດີມາສ້າງເປັນ ຂະບວນການທີ່ເປັນລະບົບ.	ຄືກັບ "ວິສະວະກອນ ໂຍທາ" ຫຼື "ສະຖາປະນິກ" ຜູ້ທີ່ນຳຄວາມຮູ້ ທາງຟີຊິກມາອອກແບບ ແລະ ວາງແຜນ ຂະບວນການທັງໝົດເພື່ອສ້າງຕຶກໃຫ້ສຳເລັດ.

ຂ. ລາຍລະອຽດຂອງແຕ່ລະສາດວິຊາ

1. ການຂຽນໂປຣແກຣມ (Programming/Coding)

- ຄຳນິຍາມ: ຄືກິດຈະກຳໃນການແປແນວຄວາມຄິດ ຫຼື ຂັ້ນຕອນວິທີ (Algorithm) ໄປເປັນຊຸດຄຳສັ່ງທີ່ຄອມພິວເຕີສາມາດເຂົ້າໃຈໄດ້ (Source Code) ໂດຍໃຊ້ພາສາຄອມພິວເຕີຕ່າງໆ.
- ເປົ້າໝາຍ: ເພື່ອສ້າງໂປຣແກຣມທີ່ສາມາດເຮັດວຽກໄດ້ຕາມທີ່ອອກແບບໄວ້.
- ຕົວຢ່າງ:
 - ການຂຽນໂຄດ Python ເພື່ອຄິດໄລ່ຄະແນນສະເລ່ຍຂອງນັກຮຽນ.
 - ການຂຽນໂຄດ JavaScript ເພື່ອເຮັດໃຫ້ປຸ່ມໃນໜ້າເວັບສາມາດກົດໄດ້.
 - ການຂຽນໂປຣແກຣມເປັນກິດຈະກຳທີ່ເນັ້ນໃສ່ການລົງມືກໍາ, ມັກຈະເປັນການເຮັດວຽກຄົນດຽວ ຫຼື ກຸ່ມນ້ອຍໆ. ມັນຄືທັກສະພື້ນຖານທີ່ສຸດທີ່ທຸກຄົນໃນສາຍງານນີ້ຕ້ອງມີ.

2. ວິທະຍາສາດຄອມພິວເຕີ (Computer Science - CS)

- ຄຳນິຍາມ: ຄືການສຶກສາໃນທາງທິດສະດີກ່ຽວກັບການຄຳນວນ (Computation) ແລະ ຂໍ້ມູນ (Information). ມັນສຶກສາເຖິງພື້ນຖານທາງຄະນິດສາດ ແລະ ຕັກກະສາດທີ່ຢູ່ເບື້ອງຫຼັງຂອງຄອມພິວເຕີ.
- ເປົ້າໝາຍ: ເພື່ອເຂົ້າໃຈທິດສະດີ, ຄຸ້ນຄວາມ, ແລະ ສ້າງແນວຄວາມຄິດໃໝ່ໆ ທີ່ສາມາດເຮັດໃຫ້ການຄຳນວນມີປະສິດທິພາບຫຼາຍຂຶ້ນ.
- ຫົວຂໍ້ທີ່ສຳຄັນ:
 - ຂັ້ນຕອນວິທີ ແລະ ໂຄງສ້າງຂໍ້ມູນ (Algorithms & Data Structures): ວິທີການແກ້ໄຂບັນຫາທີ່ມີປະສິດທິພາບທີ່ສຸດ.
 - ທິດສະດີການຄຳນວນ (Theory of Computation): ສຶກສາຂີດຈຳກັດຂອງສິ່ງທີ່ຄອມພິວເຕີສາມາດເຮັດໄດ້.

- ພາສາໂປຣແກຣມມິງ ແລະ ຄອມໄພລີ (Programming Languages & Compilers): ການອອກແບບພາສາໃໝ່ໆ ແລະ ວິທີການແປພາສາ.
- ປັນຍາປະດິດ (Artificial Intelligence - AI): ການເຮັດໃຫ້ຄອມພິວເຕີສາມາດຄິດຄືກັບມະນຸດ.
- CS ຄືການສຶກສາ "ສິ່ງທີ່ເປັນໄປໄດ້" (What is possible?) ມັນບໍ່ໄດ້ສົນໃຈໂດຍກົງກັບການສ້າງຜະລິດຕະພັນ, ແຕ່ສົນໃຈການສ້າງຄວາມຮູ້ພື້ນຖານ.

3. ວິສະວະກຳຊອບແວ (Software Engineering - SE)

- ຄຳນິຍາມ (IEEE): "ການນຳໃຊ້ວິທີການທີ່ເປັນລະບົບ, ມີລະບຽບວິໄນ, ແລະ ສາມາດວັດຜົນໄດ້ ເຂົ້າໃນການພັດທະນາ, ການດຳເນີນງານ ແລະ ການບຳລຸງຮັກສາຊອບແວ."
- ເປົ້າໝາຍ: ເພື່ອສ້າງຜະລິດຕະພັນຊອບແວທີ່ມີຄຸນນະພາບສູງ, ສາມາດສົ່ງມອບໄດ້ທັນເວລາ ແລະ ຢູ່ໃນງົບປະມານທີ່ກຳນົດ, ໂດຍສະເພາະໃນໂຄງການຂະໜາດໃຫຍ່ ແລະ ຊັບຊ້ອນທີ່ຕ້ອງເຮັດວຽກເປັນທີມ.
- ຫົວຂໍ້ທີ່ສຶກສາ:
 - ຂະບວນການພັດທະນາ (Software Process): Waterfall, Agile, Scrum, Kanban.
 - ການບໍລິຫານໂຄງການ (Project Management): ການວາງແຜນ, ການປະເມີນລາຄາ, ການຈັດການຄວາມສ່ຽງ.
 - ວິສະວະກຳຄວາມຕ້ອງການ (Requirement Engineering): ການເກັບກຳ ແລະ ວິເຄາະຄວາມຕ້ອງການ.
 - ການອອກແບບຊອບແວ (Software Design): ການອອກແບບສະຖາປັດຕະຍະກຳ ແລະ ໂຄງສ້າງຂອງລະບົບ.
 - ການຮັບປະກັນຄຸນນະພາບ (Software Quality Assurance): ການທົດສອບ, ການກວດສອບ.
- SE ຄືການສຶກສາ "ການເຮັດໃຫ້ສິ່ງທີ່ເປັນໄປໄດ້ເກີດຂຶ້ນຈິງຢ່າງມີປະສິດທິພາບ" (How to make it happen effectively?) ມັນເອົາຄວາມຮູ້ຈາກ CS ມາສ້າງເປັນຂະບວນການເພື່ອແກ້ໄຂບັນຫາໃນໂລກແຫ່ງຄວາມເປັນຈິງ.

ຄ. ສະຫຼຸບ

ຖ້າໂປຣເຈັກຄີການສ້າງແອັບ ເຄື່ອງຄິດເລກແບບງ່າຍໆ ດ້ວຍຕົວຄົນດຽວ, ຄວາມຮູ້ດ້ານ **ການຂຽນໂປຣແກຣມ** ກໍພຽງພໍ.

ຖ້າໂປຣເຈັກຄີການຄິດຄົ້ນຂັ້ນຕອນວິທີ (Algorithm) ໃໝ່ທີ່ສາມາດຄຸນເລກຈຳນວນມະຫາສານໄດ້ໄວກວ່າເດີມ, ນັ້ນຄືວຽກຂອງ **ວິທະຍາສາດຄອມພິວເຕີ**.

ແຕ່ຖ້າໂປຣເຈັກຄີການສ້າງລະບົບທະນາຄານອອນລາຍ (Online Banking) ທີ່ຕ້ອງໃຊ້ທີມງານ 50 ຄົນ, ໃຊ້ເວລາພັດທະນາ 2 ປີ, ມີງົບປະມານຈຳກັດ, ຕ້ອງຮອງຮັບຜູ້ໃຊ້ງານຫຼາຍລ້ານຄົນ ແລະ ຕ້ອງມີຄວາມປອດໄພສູງສຸດ... ນັ້ນຄື **ໂລກຂອງວິສະວະກຳຊອບແວ**.

ໃນວິຊານີ້, ເຮົາຈະໄດ້ຮຽນຮູ້ເຄື່ອງມື ແລະ ຂະບວນການຕ່າງໆ ທີ່ວິສະວະກອນຊອບແວໃຊ້ໃນການຈັດການກັບຄວາມຊັບຊ້ອນຂອງໂຄງການຂະໜາດໃຫຍ່ ເພື່ອຫຼີກລ່ຽງ "ວິກິດການຊອບແວ" ແລະ ສ້າງຜະລິດຕະພັນທີ່ປະສິດທິພາບສູງສຸດ.

1.3 ຄຸນລັກສະນະຂອງຊອບແວທີ່ດີ (Characteristics of Good Software)

ຈາກວິກິດການຊອບແວ, ເຮົາໄດ້ເຫັນຜົນຮ້າຍຂອງການສ້າງຊອບແວທີ່ບໍ່ມີຄຸນນະພາບ. ແຕ່ຄຳວ່າ "ຄຸນນະພາບ" ໃນໂລກຂອງຊອບແວໝາຍເຖິງຫຍັງແທ້? ວິສະວະກອນຊອບແວບໍ່ໄດ້ວັດແທກຄຸນນະພາບດ້ວຍຄວາມຮູ້ສຶກ, ແຕ່ຈະວັດແທກຜ່ານຄຸນລັກສະນະທີ່ຊັດເຈນ ແລະ ສາມາດປະເມີນໄດ້. ຊອບແວທີ່ດີຄວນຈະມີຄຸນລັກສະນະຫຼັກໆ ດັ່ງຕໍ່ໄປນີ້:

ກ. ຄວາມສາມາດໃນການບຳລຸງຮັກສາ (Maintainability)

- **ຄຳຖາມຫຼັກ:** ເມື່ອມີການປ່ຽນແປງເກີດຂຶ້ນ, ເຮົາສາມາດແກ້ໄຂ ຫຼື ຕໍ່ເຕີມຊອບແວໄດ້ງ່າຍປານໃດ?
- **ຄຳອະທິບາຍ:** ຄວາມສາມາດໃນການບຳລຸງຮັກສາ ຄືຄຸນລັກສະນະທີ່ສຳຄັນທີ່ສຸດໃນໄລຍະຍາວ. ມີການປະເມີນວ່າ ຫຼາຍກວ່າ 80% ຂອງຄ່າໃຊ້ຈ່າຍທັງໝົດຂອງຊອບແວ ເກີດຂຶ້ນໃນຊ່ວງ "ການບຳລຸງຮັກສາ" ຫຼັງຈາກທີ່ເປີດໃຫ້ໃຊ້ງານແລ້ວ. ຊອບແວທີ່ບຳລຸງຮັກສາງ່າຍ ຄວນຈະ:
 - **ເຂົ້າໃຈງ່າຍ:** ໂຄງສ້າງຂອງໂຄດມີລະບຽບ, ມີການຕັ້ງຊື່ຕົວແປ ແລະ ຟັງຊັນທີ່ສີ່ຄວາມໝາຍ, ແລະ ມີເອກະສານປະກອບທີ່ດີ.
 - **ດັດແກ້ງ່າຍ:** ການແກ້ໄຂ Bug ຫຼື ການເພີ່ມ Feature ໃໝ່ໃນສ່ວນໜຶ່ງ ບໍ່ຄວນສົ່ງຜົນກະທົບທີ່ບໍ່ຄາດຄິດໄປຍັງສ່ວນອື່ນໆ ຂອງລະບົບ (Low Coupling).
 - **ສາມາດປັບຕົວໄດ້:** ສາມາດປັບປ່ຽນໃຫ້ເຂົ້າກັບສະພາບແວດລ້ອມໃໝ່ໆ ໄດ້ງ່າຍ ເຊັ່ນ: ການອັບເກຣດລະບົບປະຕິບັດການ ຫຼື ການປ່ຽນຖານຂໍ້ມູນ.
- **ຕົວຢ່າງ:**
 - **Maintainability ບໍ່ດີ:** ໂປຣແກຣມເມີຄົນໜຶ່ງຂຽນໂຄດທັງໝົດ 5,000 ແຖວ ຢູ່ໃນຟາຍລຽວ, ຕັ້ງຊື່ຕົວແປວ່າ x, y, z, a, b, c. ເມື່ອໂປຣແກຣມເມີຄົນນີ້ລາອອກໄປ, ໂປຣແກຣມເມີຄົນໃໝ່ຕ້ອງໃຊ້ເວລາຫຼາຍອາທິດເພື່ອພະຍາຍາມເຂົ້າໃຈວ່າໂຄດເຮັດວຽກແນວໃດ.
 - **Maintainability ດີ:** ໂປຣແກຣມເມີອີກຄົນໜຶ່ງ ແບ່ງໂຄງສ້າງໂຄດອອກເປັນໂມດູນ (Module) ຕ່າງໆ ຕາມໜ້າທີ່, ແຕ່ລະໂມດູນມີໜ້າທີ່ຮັບຜິດຊອບຊັດເຈນ. ເຊັ່ນ: ໂມດູນຈັດການຖານຂໍ້ມູນ, ໂມດູນຈັດການການສະແດງຜິດ. ເຮັດໃຫ້ໂປຣແກຣມເມີຄົນໃໝ່ສາມາດເຂົ້າໄປແກ້ໄຂ ຫຼື ເພີ່ມເຕີມໄດ້ງ່າຍ.

ຂ. ຄວາມໜ້າເຊື່ອຖື ແລະ ໄວ້ວາງໃຈໄດ້ (Dependability and Reliability)

- **ຄຳຖາມຫຼັກ:** ເຮົາສາມາດໄວ້ວາງໃຈໃຫ້ຊອບແວນີ້ເຮັດວຽກໄດ້ຖືກຕ້ອງທຸກຄັ້ງທີ່ຕ້ອງການ ຫຼືບໍ່?
- **ຄຳອະທິບາຍ:** ຄວາມໜ້າເຊື່ອຖືບໍ່ແມ່ນພຽງແຕ່ການບໍ່ມີ Bug. ມັນປະກອບດ້ວຍຫຼາຍດ້ານ:
 - **Reliability (ຄວາມເຊື່ອຖືໄດ້):** ຊອບແວຕ້ອງເຮັດວຽກໄດ້ຢ່າງຕໍ່ເນື່ອງໂດຍບໍ່ລົ້ມເຫຼວ (crash) ພາຍໃຕ້ເງື່ອນໄຂການໃຊ້ງານປົກກະຕິ.
 - **Security (ຄວາມປອດໄພ):** ຊອບແວຕ້ອງສາມາດປ້ອງກັນຕົວເອງຈາກການເຂົ້າເຖິງ ຫຼື ການໂຈມຕີທີ່ບໍ່ໄດ້ຮັບອະນຸຍາດໄດ້.
 - **Safety (ຄວາມປອດໄພຕໍ່ຊີວິດ):** ໃນລະບົບທີ່ສຳຄັນ (critical systems), ຊອບແວຕ້ອງບໍ່ກໍ່ໃຫ້ເກີດຄວາມເສຍຫາຍຕໍ່ຊີວິດ ຫຼື ຊັບສິນ ເຖິງແມ່ນວ່າຈະມີຄວາມຜິດພາດເກີດຂຶ້ນກໍຕາມ.

- **ຕົວຢ່າງ:**

- **Dependability ບໍ່ດີ:** ແອັບ ທະນາຄານເທິງມືຖື ທີ່ມັກຈະຂັດຂ້ອງ ຫຼື ຢຸດເຮັດວຽກກະທັນຫັນ ໃນຂະນະທີ່ກຳລັງເຮັດທຸລະກຳໂອນເງິນ. ນີ້ເຮັດໃຫ້ຜູ້ໃຊ້ບໍ່ກ້າວ້ວາງໃຈ.
- **Dependability ດີ:** ລະບົບຄວບຄຸມການບິນຂອງເຮືອບິນ ທີ່ຖືກອອກແບບມາໃຫ້ມີລະບົບສຳຮອງ (redundancy) ຫຼາຍຊັ້ນ. ຖ້າລະບົບໜຶ່ງລົ້ມເຫຼວ, ອີກລະບົບໜຶ່ງຈະເຂົ້າມາເຮັດວຽກແທນທັນທີ ເພື່ອຮັບປະກັນຄວາມປອດໄພສູງສຸດ.

ຄ. ປະສິດທິພາບ (Efficiency)

- **ຄຳຖາມຫຼັກ:** ຊອບແວໃຊ້ຊັບພະຍາກອນ (CPU, Memory, Network) ໄດ້ດີປານໃດ?
- **ຄຳອະທິບາຍ:** ຊອບແວທີ່ມີປະສິດທິພາບ ຈະບໍ່ໃຊ້ຊັບພະຍາກອນຂອງລະບົບຫຼາຍເກີນຄວາມຈຳເປັນ. ມັນກ່ຽວຂ້ອງກັບ:
 - **ຄວາມໄວໃນການຕອບສະໜອງ (Response Time):** ໃຊ້ເວລາໜ້ອຍໃນການປະມວນຜົນ ແລະ ສະແດງຜົນໃຫ້ຜູ້ໃຊ້.
 - **ການໃຊ້ໜ່ວຍຄວາມຈຳ (Memory Usage):** ບໍ່ກິນ RAM ຫຼາຍເກີນໄປຈົນເຮັດໃຫ້ລະບົບທັງໝົດຊ້າລົງ.
 - **ການໃຊ້ງານ CPU:** ບໍ່ເຮັດໃຫ້ CPU ເຮັດວຽກໜັກ 100% ຕະຫຼອດເວລາໃນການເຮັດວຽກງ່າຍໆ.
- **ຕົວຢ່າງ:**
 - **Efficiency ບໍ່ດີ:** ໂປຣແກຣມແກ້ໄຂຮູບພາບທີ່ໃຊ້ເວລາ 1 ນາທີ ເພື່ອຈະໃສ່ຟິວເຕີງ່າຍໆ ໃຫ້ກັບຮູບ.
 - **Efficiency ດີ:** ໂປຣແກຣມຄົ້ນຫາຂໍ້ມູນໃນຖານຂໍ້ມູນຂະໜາດໃຫຍ່ ແຕ່ສາມາດສະແດງຜົນໄດ້ພາຍໃນ 1 ພື້ນທີ່ ພໍເທົ່າໃດວິນາທີ ເນື່ອງຈາກມີການອອກແບບຂັ້ນຕອນວິທີ (Algorithm) ທີ່ມີປະສິດທິພາບ.

ງ. ຄວາມງ່າຍໃນການໃຊ້ງານ (Usability)

- **ຄຳຖາມຫຼັກ:** ຜູ້ໃຊ້ສາມາດຮຽນຮູ້ ແລະ ໃຊ້ງານຊອບແວນີ້ໄດ້ຢ່າງສະດວກ, ວ່ອງໄວ, ແລະ ພໍໃຈ ຫຼືບໍ່?
- **ຄຳອະທິບາຍ:** ນີ້ຄືຄຸນລັກສະນະທີ່ເນັ້ນໃສ່ມຸມມອງຂອງຜູ້ໃຊ້ໂດຍກົງ. ຊອບແວອາດຈະມີ Feature ຫຼາຍ ແລະ ເຮັດວຽກໄດ້ຢ່າງມີປະສິດທິພາບ, ແຕ່ຖ້າຜູ້ໃຊ້ບໍ່ເປັນ, ມັນກໍບໍ່ມີປະໂຫຍດ. ມັນປະກອບດ້ວຍ:
 - **ຮຽນຮູ້ງ່າຍ (Learnability):** ຜູ້ໃຊ້ໃໝ່ສາມາດເລີ່ມໃຊ້ງານໄດ້ໄວໂດຍບໍ່ຕ້ອງຜ່ານການຝຶກອົບຮົມທີ່ຍາວນານ.
 - **ມີປະສິດທິຜົນ (Effectiveness):** ຜູ້ໃຊ້ສາມາດເຮັດວຽກທີ່ຕ້ອງການໃຫ້ສຳເລັດໄດ້ຢ່າງຖືກຕ້ອງ.
 - **ຄວາມພໍໃຈຂອງຜູ້ໃຊ້ (User Satisfaction):** ຜູ້ໃຊ້ຮູ້ສຶກດີ ແລະ ບໍ່ຮູ້ສຶກຫຼຸດຫງົດ ເມື່ອໃຊ້ງານຊອບແວ.
- **ຕົວຢ່າງ:**
 - **Usability ບໍ່ດີ:** ເວັບໄຊຂອງລັດຖະບານບາງແຫ່ງ ທີ່ມີເມນູຊັບຊ້ອນ, ໃຊ້ຄຳສັບທີ່ເຂົ້າໃຈຍາກ, ແລະ ຫາຂໍ້ມູນທີ່ຕ້ອງການບໍ່ເຫັນ.

- **Usability** ດີ: ແອັບ, ສິ່ງອາຫານ ທີ່ມີການອອກແບບທີ່ສະອາດຕາ, ໄອຄອນ (icon) ເຂົ້າໃຈງ່າຍ, ແລະ ມີຂັ້ນຕອນການສັ່ງຊື້ພຽງບໍ່ຄ່ຳຂັ້ນຕອນ.

ຈ. ຄວາມຖືກຕ້ອງ (Correctness) - ພື້ນຖານທີ່ສຸດ

- **ຄໍາຖາມຫຼັກ:** ຊອບແວເຮັດວຽກໄດ້ຖືກຕ້ອງຕາມທີ່ລະບຸໄວ້ໃນເອກະສານຄວາມຕ້ອງການ (Requirements) ຫຼື ບໍ່?
- **ຄໍາອະທິບາຍ:** ນີ້ຄືຄຸນລັກສະນະພື້ນຖານທີ່ສຸດທີ່ຊອບແວທຸກໂຕຕ້ອງມີ. ຖ້າຊອບແວເຮັດວຽກບໍ່ຖືກຕ້ອງ, ຄຸນລັກສະນະອື່ນໆ ກໍບໍ່ມີຄວາມໝາຍ.
- **ຕົວຢ່າງ:**
 - ຖ້າແອັບ ເຄື່ອງຄິດເລກ ຄິດໄລ່ $2 + 2$ ແລ້ວໄດ້ຜິນອອກມາເປັນ 5, ແອັບ ນັ້ນກໍຖືວ່າລົ້ມເຫຼວໂດຍສິ້ນເຊີງ, ເຖິງແມ່ນວ່າມັນຈະມີການອອກແບບທີ່ສວຍງາມ ຫຼື ເຮັດວຽກໄດ້ໄວປານໃດກໍຕາມ.

ບົດສະຫຼຸບ:

ການສ້າງຊອບແວທີ່ດີ ບໍ່ແມ່ນການເລືອກເອົາຄຸນລັກສະນະໃດໜຶ່ງ, ແຕ່ຄືການສ້າງ "**ຄວາມສົມດຸນ (Balance)**" ລະຫວ່າງ ຄຸນລັກສະນະເຫຼົ່ານີ້. ບາງຄັ້ງ, ການເພີ່ມປະສິດທິພາບອາດຈະເຮັດໃຫ້ໂຄດຊັບຊ້ອນຂຶ້ນ ແລະ ບໍາລຸງຮັກສາຍາກຂຶ້ນ. ໜ້າທີ່ຂອງວິສະວະກອນຊອບແວ ຄືການຕັດສິນໃຈວ່າຈະໃຫ້ຄວາມສໍາຄັນກັບດ້ານໃດຫຼາຍທີ່ສຸດ ເພື່ອໃຫ້ຕອບໂຈດຂອງໂຄງການນັ້ນໆ ໄດ້ດີທີ່ສຸດ.

1.4 ຈັນຍາບັນ ແລະ ໜ້າທີ່ຄວາມຮັບຜິດຊອບຂອງວິສະວະກອນຊອບແວ

ພະລັງຂອງຊອບແວໃນໂລກປັດຈຸບັນແມ່ນມີຢ່າງມະຫາສານ. ໂຄດທີ່ວິສະວະກອນຊອບແວຂຽນຂຶ້ນ ສາມາດສົ່ງຜົນກະທົບໂດຍກົງຕໍ່ການເງິນ, ຄວາມປອດໄພ, ຄວາມເປັນສ່ວນຕົວ, ແລະ ແມ່ນແຕ່ຊີວິດຂອງຜູ້ຄົນ. ດ້ວຍເຫດນີ້, ການເປັນວິສະວະກອນຊອບແວຈຶ່ງບໍ່ແມ່ນພຽງແຕ່ເລື່ອງຂອງທັກສະທາງເທັກນິກ, ແຕ່ຍັງເປັນວິຊາຊີບທີ່ຕ້ອງການຄວາມຮັບຜິດຊອບສູງ ແລະ ມີຂອບເຂດທາງຈັນຍາບັນທີ່ຊັດເຈນ, ຄືກັນກັບແພດໝໍ ຫຼື ວິສະວະກອນໂຍທາ.

ອົງກອນສາກົນທີ່ໃຫຍ່ທີ່ສຸດໃນວົງການຄອມພິວເຕີ ຄື ACM (Association for Computing Machinery) ແລະ IEEE-CS (IEEE Computer Society) ໄດ້ຮ່ວມກັນກໍານົດຫຼັກຈັນຍາບັນ 8 ປະການ ເພື່ອເປັນແນວທາງໃຫ້ວິສະວະກອນຊອບແວທຸກຄົນປະຕິບັດຕາມ. ຫຼັກການເຫຼົ່ານີ້ບໍ່ແມ່ນກົດໝາຍ, ແຕ່ເປັນມາດຕະຖານຂອງການປະພຶດຕົນແບບມີອາຊີບ.

ຫຼັກຈັນຍາບັນ 8 ປະການຂອງວິສະວະກອນຊອບແວ

1. ຜົນປະໂຫຍດສາທາລະນະ (PUBLIC)

- **ຫຼັກການ:** ວິສະວະກອນຊອບແວຕ້ອງປະຕິບັດໜ້າທີ່ຢ່າງສອດຄ່ອງກັບຜົນປະໂຫຍດຂອງສາທາລະນະຊົນ.

- **ຄຳອະທິບາຍ:** ນີ້ຄືຫຼັກການທີ່ສຳຄັນທີ່ສຸດ. ຄວາມປອດໄພ, ຄວາມເປັນສ່ວນຕົວ, ແລະ ສະຫວັດດີພາບຂອງສັງຄົມຕ້ອງມາກ່ອນຜົນປະໂຫຍດຂອງບໍລິສັດ ຫຼື ລູກຄ້າສະເໝີ.
- **ຕົວຢ່າງ:** ຫົວໜ້າສັ່ງໃຫ້ທ່ານສ້າງ Feature ທີ່ເກັບຂໍ້ມູນສ່ວນຕົວຂອງຜູ້ໃຊ້ໂດຍບໍ່ໄດ້ຮັບການຍິນຍອມ. ໃນຖານະວິສະວະກອນທີ່ມີຈັນຍາບັນ, ທ່ານຕ້ອງຄັດຄ້ານ ແລະ ຊີ້ແຈງເຖິງບັນຫາດ້ານຄວາມເປັນສ່ວນຕົວ ແລະ ຜົນກະທົບທາງລົບທີ່ອາດຈະເກີດຂຶ້ນ.

2. ລູກຄ້າ ແລະ ນາຍຈ້າງ (CLIENT AND EMPLOYER)

- **ຫຼັກການ:** ວິສະວະກອນຊອບແວຕ້ອງປະຕິບັດໜ້າທີ່ໃນທາງທີ່ເປັນຜົນປະໂຫຍດສູງສຸດຂອງລູກຄ້າ ແລະ ນາຍຈ້າງ, ໂດຍສອດຄ່ອງກັບຜົນປະໂຫຍດຂອງສາທາລະນະ.
- **ຄຳອະທິບາຍ:** ທ່ານຕ້ອງມີຄວາມຊື່ສັດຕໍ່ອົງກອນ, ຮັກສາຄວາມລັບທາງທຸລະກິດ, ແລະ ບໍ່ນຳໃຊ້ຊັບສິນຂອງບໍລິສັດເພື່ອຜົນປະໂຫຍດສ່ວນຕົວ. ທ່ານຕ້ອງມີຄວາມຊື່ສັດໃນການປະເມີນເວລາ ແລະ ຕົ້ນທຶນຂອງໂຄງການ.
- **ຕົວຢ່າງ:** ທ່ານຮູ້ວ່າໂຄງການໜຶ່ງຈະໃຊ້ເວລາພັດທະນາ 6 ເດືອນ, ແຕ່ທີມການຕະຫຼາດຕ້ອງການໃຫ້ທ່ານບອກລູກຄ້າວ່າຈະສຳເລັດພາຍໃນ 3 ເດືອນ. ການກະທຳທີ່ຖືກຕ້ອງຄືການໃຫ້ຂໍ້ມູນຕາມຄວາມເປັນຈິງ ແລະ ຊີ້ແຈງເຖິງຄວາມສ່ຽງຂອງການກຳນົດເວລາທີ່ບໍ່ເໝາະສົມ.

3. ຜະລິດຕະພັນ (PRODUCT)

- **ຫຼັກການ:** ວິສະວະກອນຊອບແວຕ້ອງຮັບປະກັນວ່າຜະລິດຕະພັນ ແລະ ການດັດແກ້ຂອງຕົນ ມີຄຸນນະພາບສູງສຸດເທົ່າທີ່ຈະເປັນໄປໄດ້.
- **ຄຳອະທິບາຍ:** ທ່ານມີໜ້າທີ່ຮັບຜິດຊອບໃນການສ້າງຊອບແວທີ່ບໍ່ພຽງແຕ່ເຮັດວຽກໄດ້, ແຕ່ຍັງຕ້ອງມີຄຸນນະພາບ, ຜ່ານການທົດສອບຢ່າງເຂັ້ມງວດ, ມີເອກະສານທີ່ດີ, ແລະ ສາມາດບຳລຸງຮັກສາໄດ້ງ່າຍ. ບໍ່ຄວນຕັດຂັ້ນຕອນທີ່ສຳຄັນ ເຊັ່ນ ການທົດສອບ ເພື່ອໃຫ້ວຽກໄວຂຶ້ນ.
- **ຕົວຢ່າງ:** ເມື່ອໃກ້ຈະຮອດກຳນົດສິ່ງມອບງານ, ຫົວໜ້າສັ່ງໃຫ້ປ່ອຍຊອບແວອອກໄປກ່ອນໂດຍບໍ່ຕ້ອງຜ່ານຂັ້ນຕອນການທົດສອບຄວາມປອດໄພ (Security Testing). ວິສະວະກອນທີ່ດີຄວນຊີ້ແຈງເຖິງຄວາມສ່ຽງຮ້າຍແຮງ ແລະ ຍືນຍັນວ່າຕ້ອງທົດສອບກ່ອນ.

4. ການຕັດສິນໃຈແບບມີອາຊີບ (JUDGMENT)

- **ຫຼັກການ:** ວິສະວະກອນຊອບແວຕ້ອງຮັກສາຄວາມຊື່ສັດ ແລະ ຄວາມເປັນອິດສະຫຼະໃນການຕັດສິນໃຈທາງວິຊາຊີບ.
- **ຄຳອະທິບາຍ:** ການຕັດສິນໃຈທາງເທັກນິກຂອງທ່ານຕ້ອງອີງໃສ່ຂໍ້ມູນ ແລະ ຫຼັກການ, ບໍ່ແມ່ນຄວາມລຳອຽງສ່ວນຕົວ ຫຼື ການຖືກກົດດັນຈາກຜູ້ອື່ນ.
- **ຕົວຢ່າງ:** ບໍລິສັດຂາຍຖານຂໍ້ມູນແຫ່ງໜຶ່ງສະເໜີໃຫ້ເງິນຄອມມິດຊັ້ນແກ່ທ່ານ ຖ້າທ່ານເລືອກໃຊ້ຜະລິດຕະພັນຂອງພວກເຂົາໃນໂຄງການ. ຈັນຍາບັນຄືການປະຕິເສດ ແລະ ເລືອກໃຊ້ເທັກໂນໂລຢີເໝາະສົມກັບໂຄງການທີ່ສຸດ, ໂດຍບໍ່ຄຳນຶງເຖິງຜົນປະໂຫຍດສ່ວນຕົວ.

5. ການບໍລິຫານຈັດການ (MANAGEMENT)

- **ຫຼັກການ:** ຜູ້ຈັດການ ແລະ ຫົວໜ້າທີມວິສະວະກຳຊອບແວ ຕ້ອງສົ່ງເສີມ ແລະ ປະຕິບັດຕາມແນວທາງການບໍລິຫານທີ່ຖືກຕ້ອງຕາມຈັນຍາບັນ.

- **ຄຳອະທິບາຍ:** ຖ້າທ່ານຢູ່ໃນຕຳແໜ່ງບໍລິຫານ, ທ່ານມີໜ້າທີ່ໃນການສ້າງສະພາບແວດລ້ອມການເຮັດວຽກທີ່ດີ, ກຳນົດຕາຕະລາງການເຮັດວຽກທີ່ສົມເຫດສົມຜົນ, ແລະ ສົ່ງເສີມການພັດທະນາຄວາມຮູ້ຂອງທີມງານ.
- **ຕົວຢ່າງ:** ຫົວໜ້າທີມທີ່ດີຈະບໍ່ສັ່ງໃຫ້ທີມງານເຮັດວຽກລ່ວງເວລາຢ່າງໜັກໜ່ວງຕະຫຼອດເວລາເພື່ອໃຫ້ທັນກຳນົດເວລາທີ່ເປັນໄປບໍ່ໄດ້. ແຕ່ຈະເຈລະຈາກັບລູກຄ້າເພື່ອຈັດການຄວາມຄາດຫວັງໃໝ່.

6. ວິຊາຊີບ (PROFESSION)

- **ຫຼັກການ:** ວິສະວະກອນຊອບແວຕ້ອງສົ່ງເສີມຄວາມຊື່ສັດ ແລະ ຊື່ສຽງຂອງວິຊາຊີບໃຫ້ສອດຄ້ອງກັບຜົນປະໂຫຍດຂອງສາທາລະນະ.
- **ຄຳອະທິບາຍ:** ທ່ານມີສ່ວນຮ່ວມໃນການເຮັດໃຫ້ວິຊາຊີບນີ້ເປັນທີ່ໜ້ານັບຖື. ນີ້ລວມເຖິງການແບ່ງປັນຄວາມຮູ້, ການຊ່ວຍເຫຼືອຜູ້ທີ່ກຳລັງເຂົ້າມາໃນວົງການ, ແລະ ການລາຍງານການກະທຳທີ່ບໍ່ຖືກຕ້ອງຕາມຈັນຍາບັນ.
- **ຕົວຢ່າງ:** ການເຂົ້າຮ່ວມໃນໂຄງການ Open Source, ການຂຽນບົດຄວາມທາງເທັກນິກ, ຫຼື ການເປັນ Mentor ໃຫ້ນັກສຶກສາຝຶກງານ.

7. ເພື່ອນຮ່ວມງານ (COLLEAGUES)

- **ຫຼັກການ:** ວິສະວະກອນຊອບແວຕ້ອງມີຄວາມຍຸຕິທຳ ແລະ ໃຫ້ການສະໜັບສະໜູນຕໍ່ເພື່ອນຮ່ວມງານ.
- **ຄຳອະທິບາຍ:** ໃຫ້ກຽດ, ຮ່ວມມື, ແລະ ໃຫ້ຄຳຕິຊົມທີ່ສ້າງສັນແກ່ເພື່ອນຮ່ວມງານ. ໃຫ້ກຽດຕິຍົດແກ່ຜົນງານຂອງຜູ້ອື່ນຢ່າງຖືກຕ້ອງ.
- **ຕົວຢ່າງ:** ໃນເວລາເຮັດ Code Review, ແທນທີ່ຈະຕຳໝາຍວ່າ "ໂຄດນີ້ຂຽນໄດ້ບໍ່ດີເລີຍ", ໃຫ້ຄຳແນະນຳທີ່ສ້າງສັນວ່າ "ລອງໃຊ້ວິທີນີ້ເບິ່ງບໍ່? ມັນອາດຈະມີປະສິດທິພາບຫຼາຍຂຶ້ນ."

8. ຕົນເອງ (SELF)

- **ຫຼັກການ:** ວິສະວະກອນຊອບແວຕ້ອງເຂົ້າຮ່ວມໃນການຮຽນຮູ້ຕະຫຼອດຊີວິດທີ່ກ່ຽວຂ້ອງກັບການປະຕິບັດວິຊາຊີບຂອງຕົນ ແລະ ສົ່ງເສີມແນວທາງປະຕິບັດທີ່ຖືກຕ້ອງຕາມຈັນຍາບັນ.
- **ຄຳອະທິບາຍ:** ເທັກໂນໂລຢີປ່ຽນແປງໄວຫຼາຍ. ທ່ານມີໜ້າທີ່ຈັນຍາບັນໃນການພັດທະນາຄວາມຮູ້ຂອງຕົນເອງຢູ່ສະເໝີ ເພື່ອໃຫ້ສາມາດສ້າງຊອບແວທີ່ມີຄຸນນະພາບ ແລະ ທັນສະໄໝໄດ້.
- **ຕົວຢ່າງ:** ການໃຊ້ເວລາຮຽນຮູ້ພາສາໂປຣແກຣມມິງໃໝ່ໆ, ການຕິດຕາມຂ່າວສານດ້ານຄວາມປອດໄພທາງໄຊເບີ, ຫຼື ການເຂົ້າຮ່ວມຫຼັກສູດອອນລາຍຕ່າງໆ.

ບົດສະຫຼຸບ:

ຈັນຍາບັນບໍ່ແມ່ນສິ່ງທີ່ເລືອກປະຕິບັດໄດ້, ແຕ່ມັນຄື **ພື້ນຖານຂອງການເປັນວິສະວະກອນຊອບແວມີອາຊີບ**. ມັນຄືສິ່ງທີ່ສ້າງຄວາມໄວ້ວາງໃຈຈາກສັງຄົມ, ລູກຄ້າ, ແລະ ເພື່ອນຮ່ວມງານ. ການຕັດສິນໃຈທີ່ທ່ານເຮັດໃນທຸກໆມື້, ບໍ່ວ່າຈະນ້ອຍຫຼືໃຫຍ່, ລ້ວນແຕ່ສະທ້ອນເຖິງຈັນຍາບັນ ແລະ ຄວາມຮັບຜິດຊອບຂອງທ່ານຕໍ່ວິຊາຊີບນີ້.